

前端permit2集成方案

背景

在调用合约 `sendRealisedToken`、`encash`、`transfereeAcceptWithFN` 这三个方法时，需要传入 token 授权转账的签名结果（r、s、v）提供给合约进行转账，但本次接入的USDT稳定币合约没有 permit方法进行该签名授权转账，因此使用uniswap的permit2方案进行集成，对这三个场景进行改造。

工作流程

1. 首次授权

调用授权查询，判断已授权金额是否大于本次需要permit的金额

```
USDT.allowance(myAddress, permit2Address)
```

如果授权金额不够，用户必须调用ERC20代币合约授权Permit2合约：

```
USDT.approve(permit2Address, totalAmount);
```

为了实现一次授权无限次permit转账，用户应该对permit2合约进行最大程度的批准，金额为uint256类型的最大值，这样后续就无限重复approve操作了：

```
totalAmount = ethers.constants.MaxUint256;
```

2. 获取permit2Nonce

调用config合约的Permit2Nonce方法，获取一个nonce，供后续permit2签名使用，这块与之前直接调用token合约的nonce方法获取不同，需改造

```
const permit2Nonce = await ConfigC.PermittedNonce(myAddress);
```

3. 构建 EIP-712 PermitTransferFrom 授权结构体

```
{
  "types": {
    "EIP712Domain": [
      { "name": "name", "type": "string" },
      { "name": "chainId", "type": "uint256" },
      { "name": "verifyingContract", "type": "address" }
    ],
    "PermitTransferFrom": [
      { "name": "permitted", "type": "TokenPermissions" },
      { "name": "spender", "type": "address" },
      { "name": "nonce", "type": "uint256" },
      { "name": "deadline", "type": "uint256" }
    ]
  }
}
```

```

    ],
    "TokenPermissions": [
      { "name": "token", "type": "address" },
      { "name": "amount", "type": "uint256" }
    ]
  },
  "primaryType": "PermitTransferFrom",
  "domain": {
    "name": "Permit2",
    "chainId": 1, // 获取
    当前网络的chainId
    "verifyingContract": "0x00000000022D473030F116dDEE9F6B43aC78BA3" //
    permet2合约地址
  },
  "message": {
    "permitted": {
      "token": "0xTokenAddressHere", //
      token地址
      "amount": 1000000000000000000 //
      token金额
    },
    "spender": "0xSpenderAddressHere", // 调用
    业务合约的地址，如发起send时填dtt合约
    "nonce": 0, // 步骤
    2获取的nonce
    "deadline": 1750000000 // 过期
    时间
  }
}

```

参考脚本

```
js permit2.js × Encash.sol SendFacet.sol UserPermission.sol RorMarket.sol RorEnhanceme
23 const permit2Nonce = await ConfigC.Permmit2Nonce(signer.getAddress());
24 const deadline : number = dayjs().add( value: 1, unit: "day").unix();
25 const domain : {...} = {
26   name: "Permit2",
27   chainId: hre.network.config.chainId,
28   verifyingContract: address["Permit2"].address,
29 };
30
31 const types : {PermitTransferFrom: [{name: string, type: string},...], TokenPermissions: [{name: string, type: string},...]} = {
32   PermitTransferFrom: [
33     { name: "permitted", type: "TokenPermissions" },
34     { name: "spender", type: "address" },
35     { name: "nonce", type: "uint256" },
36     { name: "deadline", type: "uint256" },
37   ],
38   TokenPermissions: [
39     { name: "token", type: "address" },
40     { name: "amount", type: "uint256" },
41   ],
42 };
43 const value : bigint = ethers.parseUnits( value: "1.0", unit: 18); // 1 token
44 const message : {...} = {
45   permitted: {
46     token: tokenAddress,
47     amount: value,
48   },
49   spender: address["Encash"].address,
50   nonce: permit2Nonce,
51   deadline: deadline,
52 };
53
54 const signature : string = await signer.signTypedData(domain, types, message);
55 console.log("signature: ", signature);
```

[permit2.js](#)

[approveMax.js](#)

备注

permit2介绍:

<https://learnblockchain.cn/article/5161>

<https://docs.uniswap.org/contracts/permit2/overview>

Permit2的nonce为位图结构:

Uniswap 的 Permit2 没有为每个用户维护一个简单的递增 `uint256` nonce, 而是将 nonce 分割成一组 **256 位的位图 (bitmaps)**, 每个位图用一个 `uint256` 表示。

每个用户的 nonce 存在如下形式:

```
nonce(owner, wordPos) -> uint256 bitmap
```

- `wordPos` 是第几个 256-bit "word"

- 每个 word 有 256 个 bit, 分别表示第 0 ~ 255 个 nonce 是否使用
- 整体 nonce 是全局连续编号: 0, 1, 2, ..., N

这个nonce就是:

```
nonce = word 编号 × 每个 word 的长度 + 当前 bit 位的位置  
       = wordPos × 256 + bit
```

举个例子:

wordPos	bit	nonce
0	0	0
0	123	123
1	0	256
2	17	529

这个 nonce 就是你要用来签名的值 (permit.nonce) ——它唯一标识这笔授权是否已被用过。

该设计原因

1. **节省 gas**: 位图比每次更新一个 uint256 更省空间和 gas。
2. **支持并发**: 可以一次性授权多个 nonce (位) 并发操作。
3. **高吞吐量**: 适合大量使用签名授权的 DEX 类产品。